

day13\5_destructuring.js

```
1 // destructuring of an array or object
2 //destructuring means getting the values individually
3 //destructuring means to break it up
4
5 let ob1 = {
6     name: "Ding Liren",
7     money: 550000,
8     zipcode: 4140004,
9     address: "17-plank road"
10 }
11
12 //store the properties of ob1 into objy object using spread operator:
13 const {name: personName, money: personMoney, ...objy} = ob1;
14
15 console.log(personName, personMoney, objy);
16
17
18
19 //Destructuring with Array:
20 const arr = [3,2,5,7,9,11];
21 const [firstValue, secondValue, ...ary] = arr; //storing the rest of the elements of arr
    into the ary
22 console.log(firstValue, secondValue, ary);
23 const [ ,index1value, ,index3value, ,index5value] = arr;
24 console.log(index1value, index3value, index5value);
25 const [index0value, ,index2value, ,index4value, ,index6value] = arr;
26 console.log(index0value, index2value, index4value, index6value);
27
28
29 //code snippet
30 const oby = {
31     name: "tian",
32     //nested object
33     address:{
34         street: "14-food bridge",
35         landmark: "prism hotel"
36     }
37 }
38
39 const {name:humanName, address: personAddress}=oby; //here, personAddresss is an nested
    object. And we can destructure this object too using curly braces.
40 console.log(personAddress);//printing whole address
41 //destructuring the address
42 const { landmark } = personAddress;
43 const { street } = personAddress;
44 console.log(humanName, landmark);
45 console.log(humanName, street);
46
47 //another way to do above destructuring task:
48 // const {name: fullname, address:{street, landmark}} = oby;
49 // console.log(street);
50 // console.log(landmark);
```

```
51 |
52 |
53 | //code snippet
54 | const objt = {
55 |     name: "tarun",
56 |     arr:[25, 89, 75]
57 | }
58 | const{arr: arr2} = objt;
59 | console.log(arr2);
60 | const{arr: [ ,secondelement]} = objt; //doing destructuring of arr array to fetch the
    | second element
61 | console.log(secondelement);
```